Chuleta de la UT6 — WebSockets, GraphQL y documentación

WebSockets

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-websocket</artifactId>
</dependency>
```

```
@Component
public class FunkosWebSocketHandler extends
TextWebSocketHandler {

    private final Set<WebSocketSession> sesiones = new
CopyOnWriteArraySet<>();

    @Override public void
afterConnectionEstablished(WebSocketSession s)
{ sesiones.add(s); }
    @Override public void
afterConnectionClosed(WebSocketSession s, CloseStatus st)
{ sesiones.remove(s); }
    @Override public void
handleTransportError(WebSocketSession s, Throwable e)
{ sesiones.remove(s); }

    @Override protected void
handleTextMessage(WebSocketSession s, TextMessage m) { … }  //
bidireccional

    public void enviarATodos(String mensaje) {
        for (var s : sesiones) {
            try {
                if (s.isOpen()) s.sendMessage(new
TextMessage(mensaje));
                else sesiones.remove(s);
            } catch (IOException e) { sesiones.remove(s); }
        }
    }
}
```

```
@Configuration
@EnableWebSocket
public class WebSocketConfig implements WebSocketConfigurer {
    @Override public void
registerWebSocketHandlers(WebSocketHandlerRegistry registry) {
        registry.addHandler(handler, "/ws/v1/funkos")
                .setAllowedOrigins("http://localhost:
5173");    // NUNCA "*"
    }
}
```

AVISO `CopyOnWriteArraySet`, no `HashSet`: se recorre y se modifica desde hilos distintos. **AVISO** Quitar la sesión **en los tres sitios** y además al fallar el envío. **AVISO** El `try` **dentro** del bucle: sin él, un cliente roto deja sin mensaje a los demás. **AVISO** Sin `setAllowedOrigins`, cualquier web del mundo puede conectarse: los WebSockets **no** están sujetos a la política del mismo origen.

Notificar después de confirmar

```
public record FunkoCambiadoEvent(Tipo tipo, FunkoResponse
funko) {
    public enum Tipo { CREATE, UPDATE, DELETE }
}
```

```
// En el servicio: 2 líneas y nada más
private final ApplicationEventPublisher eventos;
eventos.publishEvent(new FunkoCambiadoEvent(Tipo.CREATE,
respuesta));
```

```
// En el notificador
@TransactionalEventListener(phase =
TransactionPhase.AFTER_COMMIT)
public void alCambiar(FunkoCambiadoEvent e)
{ handler.enviarATodos(json(e)); }
```

AVISO Sin `AFTER_COMMIT`**, un** `rollback` **deja el mensaje ya enviado**: los clientes ven algo que no existe. **AVISO** Se manda el **DTO**, no la entidad: aquí estamos fuera de toda transacción → `LazyInitializationException`.

	Dirección	Reconexión	Cuándo
Polling	Cliente pregunta	—	Nunca, si hay alternativa
SSE	Servidor → cliente	**Automática**	Notificaciones, progreso
WebSocket	**Bidireccional**	La haces tú	Chat, pujas, alta frecuencia

```
const ws = new WebSocket(`ws://${location.host}/ws/v1/
funkos`);
ws.onopen    = () => …;
ws.onmessage = e  => console.log(JSON.parse(e.data));
ws.onclose   = () => …;      // aquí va el reintento
ws.send(JSON.stringify({…}));
```

`HTTP/1.1 101 Switching Protocols` — el único `1xx` que verás. La conexión deja de ser HTTP.

GraphQL

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-graphql</artifactId>
</dependency>
<dependency>
  <groupId>org.springframework.graphql</groupId>
  <artifactId>spring-graphql-test</artifactId><scope>test</
scope>
</dependency>
```

src/main/resources/graphql/schema.graphqls

```
type Funko {
    id: ID!
    nombre: String!
    precio: Float!
    categoria: Categoria!
}

input FunkoInput  { nombre: String!  precio: Float!
categoria: String! }
input FunkoFiltro { categoria: String  precioMin: Float
nombre: String }

type Query {
    funkos(filtro: FunkoFiltro, pagina: Int = 0, tamano: Int =
20): [Funko!]!
    funkoById(id: ID!): Funko
}

type Mutation {
    crearFunko(input: FunkoInput!): Funko!
    borrarFunko(id: ID!): Boolean!
}

type Subscription { funkoCambiado: Notificacion! }
```

Tipo	Significa
`String`	Puede ser `null`
`String!`	No puede ser `null`
`[String]`	Lista y elementos pueden ser `null`
`[String!]!`	**Ni la lista ni los elementos** — lo correcto para un listado

Escalares: `Int` `Float` `String` `Boolean` `ID`

```
@Controller
public class FunkosGraphQlController {

    private final FunkosService servicio;        // ← EL MISMO
que usa REST

    @QueryMapping
    public List<FunkoResponse> funkos(@Argument
Optional<FunkoFiltro> filtro) { … }

    @QueryMapping
    public FunkoResponse funkoById(@Argument Long id) { return
servicio.findById(id); }

    @MutationMapping
    public FunkoResponse crearFunko(@Argument("input") @Valid
FunkoCreateRequest input) { … }

    /** Campo calculado que no existe en la entidad */
    @SchemaMapping(typeName = "Matricula", field = "aprobada")
    public Boolean aprobada(MatriculaResponse m) { … }

    /** Resuelve la relación para TODOS de golpe: evita el N+1
*/
    @BatchMapping(typeName = "Funko")
    public Map<FunkoResponse, CategoriaResponse>
categoria(List<FunkoResponse> funkos) { … }

    @SubscriptionMapping
    public Flux<Notificacion> funkoCambiado() { return
emisor.asFlux(); }
}
```

```
@Component
public class GraphQlExceptionHandler extends
DataFetcherExceptionResolverAdapter {
    @Override
    protected GraphQLError resolveToSingleError(Throwable ex,
DataFetchingEnvironment env) {
        return switch (ex) {
            case FunkoNotFoundException e -> error(env,
ErrorType.NOT_FOUND, e);
            case FunkoConflictException e -> error(env,
ErrorType.BAD_REQUEST, e);
            default -> null;
        };
    }
}
```

`ErrorType`: `NOT_FOUND` · `BAD_REQUEST` · `UNAUTHORIZED` · `FORBIDDEN` · `INTERNAL_ERROR`

```
spring.graphql.graphiql.enabled=true                 # /
graphiql — SOLO dev
spring.graphql.schema.printer.enabled=true           # /
graphql/schema
spring.graphql.schema.introspection.enabled=false    # false
en PROD
```

```
@Bean
public GraphQlSourceBuilderCustomizer limites() {
    return b -> b.configureGraphQl(g ->
g.instrumentation(List.of(
            new MaxQueryDepthInstrumentation(10),
            new MaxQueryComplexityInstrumentation(200))));
}
```

AVISO El N+1 es el problema característico de GraphQL. Pedir la categoría de 100 funkos son 101 consultas sin `@BatchMapping`, y **no se arregla con** `JOIN FETCH` porque la consulta depende de lo que pida el cliente. **AVISO GraphQL devuelve 200 casi siempre.** Los errores van en `errors`, así que no se puede monitorizar por el código de estado. **AVISO** Todo va por `POST` **a** `/graphql`, también las lecturas → **la caché HTTP no funciona**. **AVISO** `introspection` en producción regala el esquema completo. **AVISO** Sin límites de profundidad, **el cliente decide el coste** de tu consulta.

```
curl -s localhost:8080/graphql -H "Content-Type: application/
json" \
     -d '{"query":"{ funkos { nombre } }"}' | jq
```

```
@SpringBootTest @AutoConfigureGraphQlTester
class Test {
    @Autowired GraphQlTester tester;

    @Test void consulta() {
        tester.document("{ funkos { nombre } }").execute()
              .path("funkos[0].nombre").entity(String.class).isN
    }
    @Test void error() {
        tester.document("{ funkoById(id: 9999)
{ nombre } }").execute()
              .errors().expect(e -> e.getErrorType() ==
ErrorType.NOT_FOUND);
    }
}
```

	REST	GraphQL
Endpoints	Uno por recurso	**Uno**: `/graphql`
Qué llega	Lo que decide el servidor	**Lo que pide el cliente**
Varios recursos	Varias peticiones	**Una**
Códigos de estado	200/201/400/404/409	**200 casi siempre**
Caché HTTP	**Nativa**	Hay que montarla
Esquema	OpenAPI, opcional	**Obligatorio y verificado**
Coste de una petición	Conocido	**Lo decide el cliente**

CORS

```
@Configuration
public class CorsConfig implements WebMvcConfigurer {
    @Override public void addCorsMappings(CorsRegistry
registry) {
        registry.addMapping("/api/**")
                .allowedOrigins(origenes)        // de
application-{perfil}.properties
                .allowedMethods("GET","POST","PUT","PATCH","DELE
                .allowedHeaders("*")
                .exposedHeaders("Location")      // para que
el JS lea el 201
                .allowCredentials(true)
                .maxAge(3600);
    }
}
```

```
# dev
cors.origenes-permitidos=http://localhost:5173,http://
localhost:3000
# prod
cors.origenes-permitidos=https://miapp.es
```

El *preflight*:

```
OPTIONS /api/v1/funkos
Origin: http://localhost:5173
Access-Control-Request-Method: POST
Access-Control-Request-Headers: content-type

→ 200
Access-Control-Allow-Origin: http://localhost:5173
Access-Control-Allow-Methods: POST
Access-Control-Allow-Headers: content-type
Access-Control-Max-Age: 3600
```

Dispara *preflight*: verbo distinto de GET/HEAD/POST · `Content-Type: application/json` · cabeceras propias. Es decir: **casi cualquier petición de una API REST**.

AVISO Bloquea el NAVEGADOR, no tu servidor. La petición llega, se procesa y se responde 200; el navegador le niega el resultado al JavaScript. Por eso funciona con `curl`. **AVISO CORS no protege tu API**: protege al usuario de que otra web use su sesión. Cualquiera puede llamarte desde fuera de un navegador. **AVISO** `allowedOrigins("*")` + `allowCredentials(true)` → **el navegador lo rechaza**, y a propósito. Usa `allowedOriginPatterns`. **AVISO** Sin `exposedHeaders`, el JS solo puede leer seis cabeceras: `Location` no está entre ellas.

```
curl -i -X OPTIONS localhost:8080/api/v1/funkos \
  -H "Origin: http://localhost:5173" \
  -H "Access-Control-Request-Method: POST"
```

OpenAPI y Swagger

```
<dependency>
  <groupId>org.springdoc</groupId>
  <artifactId>springdoc-openapi-starter-webmvc-ui</artifactId>
  <version>2.7.0</version>
</dependency>
```

`/v3/api-docs`	El JSON de la especificación
`/swagger-ui/index.html`	La interfaz, **con botón de disparar**

```
@Configuration
public class OpenApiConfig {
    @Bean public OpenAPI apiInfo() {
        return new OpenAPI()
            .info(new Info().title("API de
Funkos").version("v1")
                .description("…")
                .contact(new Contact().name("…").email("…"))
                .license(new License().name("CC BY-NC-SA
4.0").url("…")))
            .servers(List.of(new Server().url("http://
localhost:8080").description("Dev")))
            .tags(List.of(new
Tag().name("Funkos").description("CRUD de funkos")));
    }
}
```

```
@Tag(name = "Funkos", description = "CRUD y búsqueda")
@RestController
public class FunkosRestController {

    @Operation(summary = "Obtiene un funko por su id",
               description = "Los borrados lógicamente se
tratan como inexistentes.")
    @ApiResponses({
        @ApiResponse(responseCode = "200", description =
"Encontrado",
            content = @Content(schema = @Schema(implementation
= FunkoResponse.class))),
        @ApiResponse(responseCode = "404", description = "No
existe", content = @Content),
        @ApiResponse(responseCode = "409", description =
"Nombre repetido", content = @Content)
    })
    @GetMapping("/{id}")
    public ResponseEntity<FunkoResponse> findById(
            @Parameter(description = "Id del funko", example =
"1", required = true)
            @PathVariable Long id) { … }
}
```

```
@Schema(description = "Datos de un funko")
public record FunkoResponse(
        @Schema(description = "Identificador", example = "1")
Long id,
        @Schema(description = "Nombre", example = "Mickey
Mouse") String nombre) {}
```

```
# dev
springdoc.api-docs.enabled=true
springdoc.swagger-ui.enabled=true
springdoc.swagger-ui.path=/swagger-ui.html
springdoc.swagger-ui.operationsSorter=method
# PROD
springdoc.api-docs.enabled=false
springdoc.swagger-ui.enabled=false
```

Lo deduce de tu código	Hay que escribirlo
Ruta, verbo, parámetros	**Para qué sirve**
Tipos y obligatoriedad (de `@NotBlank`, `@Min` …)	**Qué errores** devuelve y cuándo
Esquema de los DTOs	**Ejemplos** de valores reales

AVISO Swagger UI es un cliente HTTP completo, no un visor. En producción, apagado: publica todos tus endpoints y un botón para llamarlos. **AVISO** Los **409 de negocio** no los puede deducir ninguna herramienta. Son justo lo que necesita quien consume tu API.

Los diez errores del examen

	Síntoma	Causa
1	`ConcurrentModificationException` en el *broadcast*	`HashSet` en vez de `CopyOnWriteArr`
2	Un cliente desconectado deja sin mensaje a los demás	Falta el `try` **dentro** del bucle
3	Los clientes ven algo que no está en la BD	Falta `@TransactionalEventListener(AF`
4	`LazyInitializationException` en el notificador	Se manda la entidad, no el DTO
5	101 consultas en una query GraphQL	Falta `@BatchMapping`
6	La monitorización no detecta errores de GraphQL	GraphQL devuelve **200** casi siempre
7	Funciona en Postman y falla en el navegador	**CORS**: falta el origen permitido
8	`OPTIONS` devuelve 403	Origen no incluido en `allowedOrigi`
9	El JS no puede leer la cabecera `Location` del 201	Falta `exposedHeaders("Location")`
10	Swagger accesible en producción	Falta apagarlo en `application-prod.properties`